Depo Robotu Simülatörü

Arda Özteriş (250501031) · Ahmet Bilgekağan Karaca (250501030)

Kocaeli Sağlık ve Teknoloji Üniversitesi

YZM217/BLM217 Programlama Lab I · 2026-2027 Güz

Özet

Bu çalışmada, metin tabanlı depo haritasında bir robotun hareketini, kutu taşımasını ve enerji kullanımını modelleyen bir Python uygulaması geliştirilmiştir. Harita TXT, komutlar TXT ve ayarlar JSON dosyalarından okunur. Komutlar doğrulanarak sırayla işlenir; robotun son durumu JSON, komut geçmişi CSV olarak kaydedilir. Tkinter arayüzü aynı motoru kullanarak komutları adım adım veya zamanlanmış biçimde gösterir. Bonus GIT komutu için genişlik öncelikli arama uygulanmıştır. Sağlanan altı örnek testin tamamı ve eklenen 27 otomatik kontrol geçmiştir. Beş deneme veri setinde adımlı ve toplu yürütmenin eşitliği doğrulanmıştır. Bulgular verilen örnekler ve yerel kontrollerle sınırlıdır; daha sonra verilecek demo paketini kapsamaz.

Anahtar kelimeler: robot simülasyonu, Python, Tkinter, dosya işleme, BFS, yazılım testi.

I. Giriş

Depo robotlarının davranışı, konum ve yön gibi mekânsal bilgilerin yanında yük ve enerji durumuna bağlıdır. Bir hareketin geçerli olması yalnızca hedefin boş olmasına değil, robotun o hareketin maliyetini karşılayabilmesine de bağlıdır. Bu nedenle simülatörün temel problemi, komutların durum üzerinde oluşturduğu değişiklikleri açık ve tutarlı kurallarla hesaplamaktır.

Çalışmanın amacı, ders yönergesindeki tek robot modelini standart Python kütüphanesiyle gerçekleştirmektir. İki boyutlu listelerle harita temsili, dosya ayrıştırma, sözlüklerle durum saklama, hata yönetimi ve olay temelli arayüz programlama birlikte kullanılmıştır. Fiziksel robot denetimi, sensör belirsizliği ve çok robotlu planlama kapsam dışındadır.

Kurallar ile sunum katmanı ayrı modüllerde tutulmuştur. Motor, pencere açmadan çalışabilir. Arayüz, aynı komut uygulama fonksiyonunu çağırır. Böylece bir batarya kuralı değişikliği hem otomatik testlere hem kullanıcı ekranına aynı yerden yansır. README, kurulum ve kullanım bilgilerini içerir.

II. Yöntem

A. Mimari ve veri akışı

Uygulama iki ana modülden oluşur. simulasyon.py, girdi doğrulama, durum oluşturma, komut yürütme, yol arama ve çıktı üretimini içerir. arayuz.py, Tkinter bileşenlerini ve kullanıcı olaylarını yönetir. Komut satırı çalıştırması ve grafik arayüz aynı kuralları paylaşır. Motorun Tkinter bağımlılığı yoktur.

Harita okunduktan sonra başlangıç koordinatı ve zemin ızgarası ayrılır. Ayarlar, maliyetler ve başlangıç yönünü belirler. Komut ayrıştırıcısı geçerli komut listesini ve hatalı satır listesini oluşturur. yeni_durum, girdilerin bağımsız kopyasını alır. Her komut bu sözlüğü günceller; durum_sonucu yalnızca dışarı aktarılacak özeti çıkarır.

B. Harita ve robot durumu

Harita, satırlardan oluşan iki boyutlu listedir. Bir hücreye izgara[y][x] ile erişilir. Koordinatlar (x, y) biçimindedir; sol üst köşe (0, 0) kabul edilir. Duvar, boş alan, kutu, teslim ve şarj hücreleri sırasıyla #, ., K, T ve S ile temsil edilir. R başlangıç işaretidir; okunduktan sonra zemin boş hücreye dönüştürülür.

Robotun konumu ayrı tutulduğu için teslim ve şarj noktaları robot üzerlerinden geçerken kaybolmaz. Yük durumu mantıksal değerdir ve aynı anda en fazla bir kutu taşınabilir. Enerji, yön, teslim sayısı, çarpışma sayısı ve toplam hareket sayısı durum sözlüğünün diğer bileşenleridir. Ziyaret sözlüğünde anahtar olarak değişmez koordinat çiftleri kullanılır.

C. Girdi doğrulama

Harita boşsa, satır uzunlukları farklıysa, R sayısı bir değilse veya tanımsız sembol varsa ValueError üretilir. Ayarlarda zorunlu alanlar aranır; sayısal değerler negatif olmayan tam sayılarla sınırlandırılır. Yük çarpanı en az birdir. Bozuk JSON, kullanıcıya anlaşılır hata olarak bildirilir.

Komutlar büyük harfe duyarlıdır. ILERI bir pozitif tam sayı, GIT iki negatif olmayan tam sayı alır. Diğer komutlar parametre almaz. Hatalı satırlar atlanırken özgün satır numarası, içerik ve gerekçe korunur. Boş satırlar hata değildir ancak satır numaralandırmasına dahildir. UTF-8 BOM içeren dosyalar da okunabilir.

II. Yöntem devamı

D. Hareket ve enerji modeli

ILERI n komutu, n ayrı hareket denemesi olarak işlenir. Önce robotun baktığı yönün vektörü mevcut koordinata eklenir. Hedef harita dışındaysa veya duvarsa robot yerinde kalır, çarpışma sayısı bir artar ve çarpışma cezası uygulanır. Komutun kalan hareket denemeleri iptal edilir. Engel durumunda ayrıca adım maliyeti alınmaz.

Hedef geçilebilirse hareket maliyeti hesaplanır. Yüksüz robotun maliyeti adim_maliyeti, yüklü robotunki adim_maliyeti ile yuk_carpani çarpımıdır. Yeterli enerji varsa batarya azaltılır, konum güncellenir ve ziyaret sayısı artırılır. Yetersizse o hareket yapılmaz; önceki başarılı hareketler korunur ve simülasyon durdurulur.

Maliyet ile bataryanın eşitliği ayrı ele alınır. İşlem gerçekleştirilip batarya sıfıra iner; sonraki sıfır maliyetli işlemler mümkün kalır. Örneğin robot son enerjisiyle şarj istasyonuna ulaşmışsa SARJ çalıştırılabilir. Maliyeti karşılayamayan bir işlemden sonra ise şarj dahil sonraki komutlar çalıştırılmaz. Böylece sıfır enerji ile simülasyonun durmuş olması farklı durumlar olarak modellenir.

Çarpışma cezası bataryadan büyükse enerji sıfıra sabitlenir ve simülasyon durur. Eşitlikte enerji sıfır olur ancak durma bayrağı açılmaz. Enerji yetersizliği nedeniyle yapılamayan normal bir işlem, mevcut bataryayı değiştirmez. Sonuçtaki batarya_bitti alanı, bu nedenle durmuş olmayı belirtir.

E. Yön ve kutu işlemleri

Yönler saat yönünde K, D, G, B sırasıyla tutulur. Sağ dönüş indeksi bir artırır, sol dönüş bir azaltır; dört ile mod alma işlemi döngüyü sağlar. Dönüşten önce enerji denetlenir. Yetersiz enerjide yön değişmez. Her başarılı dönüş, donus_maliyeti kadar enerji tüketir.

AL yalnızca kutu üzerindeyken ve robot boşken geçerlidir. Alınan kutunun hücresi boş zemine çevrilir. BIRAK, teslim noktasında teslim sayısını artırır; boş zeminde yeni bir kutu oluşturur. Mevcut kutu veya şarj hücresi üzerine bırakılmaz. SARJ yalnızca S hücresinde bataryayı kapasiteye doldurur. Bu üç komut enerji harcamaz. Uygun olmayan durumda GECERSIZ olayı kaydedilir.

F. Komut işleme algoritması

Aşağıdaki yalancı kod, hareket sırasında oluşan kısmi yürütmeyi ve simülasyon durduğunda sonraki komutların atlanmasını özetler. Arayüzdeki Adım düğmesi bir satırlık komutun tamamını uygular; komut içindeki fiziksel adımlar ayrıca günlük satırı oluşturmaz.

```
durum = yeni_durum(girdiler)
her geçerli komut için:
    durum durmuşsa döngüyü bitir
    komut ILERI n ise:
        n kez:
            hedefi hesapla
            hedef engelliyse:
                çarpışma cezasını uygula
                bu komutu bitir
            enerji yetersizse:
                simülasyonu durdur
                bu komutu bitir
            enerjiyi azalt, hareket et
            ziyaret ve adımı artır
    diğer komutun kuralını uygula
    bir günlük kaydı ekle
sonuç özetini ve günlüğü yaz
```

Şekil 1. Durum temelli komut yürütmenin yalancı kodu.

G. Ziyaret ve günlük modeli

Başlangıç hücresinin ziyaret sayısı birdir. Her başarılı hareket, girilen hücrenin sayısını bir artırır. Dönüş, kutu işlemi, şarj veya çarpışma ziyaret oluşturmaz. Bütün çalışma boyunca ziyaretlerin toplamı, toplam fiziksel adımın bir fazlasıdır. Bu ilişki rastgele senaryolarda kontrol edilen temel değişmezlerden biridir.

En çok ziyaret edilen hücre seçilirken önce ziyaret sayısı büyük olan alınır. Eşitlikte küçük y, sonra küçük x tercih edilir. Uygulamada (-sayı, y, x) anahtarıyla minimum seçimi yapılır. Sonuç, koordinat ve ziyaret sayısı olarak dışarı aktarılır.

CSV günlüğü adim, komut, x, y, yon, batarya ve olay sütunlarını içerir. Buradaki adim, işlenen komutun sırasıdır; toplam_adim fiziksel hareket sayısıdır. Bir ILERI veya GIT komutu tek kayıt üretir. Ayrıştırılamayan satırlar günlük yerine hata listesine yazılır. CSV dosyası newline boş değeriyle açılır; bu, Windows'ta istenmeyen boş satırların oluşmasını önler [4].

III. Yol bulma ve arayüz

A. Genişlik öncelikli arama

Bonus GIT x y komutu, hedefe en az sayıda yatay veya dikey hücre adımıyla ulaşan yolu bulur. Her komşu geçişi bir birim uzaklık kabul edildiğinden genişlik öncelikli arama kullanılır. Başlangıç kuyruğa eklenir; sıradaki hücre çıkarılıp görülmemiş geçilebilir komşuları kuyruğun sonuna eklenir. Kuyruk, iki ucunda hızlı işlem sağlayan collections.deque ile gerçekleştirilmiştir [2].

Her hücrenin öncülü onceki sözlüğünde saklanır. Hedefe ulaşılınca öncüller geriye izlenir ve oluşan liste ters çevrilir. Hücreler ilk keşifte işaretlendiği için tekrar tekrar kuyruğa alınmaz. Komşular K, D, G, B sırasıyla incelenir; eşit uzunluklu yollar arasında seçim bu sıraya göre belirlenir. H hücreli haritada zaman ve bellek kullanımı O(H) düzeyindedir.

Bulunan yol doğrudan koordinat atamasıyla uygulanmaz. Robot her komşu hücreye doğru döner ve mevcut hareket yardımcısıyla bir adım atar. Böylece dönüş maliyeti, yük çarpanı, enerji sınırı ve ziyaret sayımı normal komutlarla ortak kalır. 180 derece yön değişiminde iki sağ dönüş kullanılır. Enerji yolun ortasında yetmezse o ana kadar gerçekleşmiş işlemler korunur.

Bu yöntem en kısa hücre yolunu arar; minimum enerji yolu bulduğu iddia edilmez. Dönüş maliyetini de eniyileyen bir çözüm, arama durumuna yön bilgisinin eklenmesini ve ağırlıklı bir yol algoritmasını gerektirebilir. Şartnamedeki bonus için eşit ağırlıklı hücre yolu tercih edilmiştir.

B. Uç durumlar

Hedef duvarsa, harita dışındaysa veya erişilemiyorsa YOL_YOK olayı oluşur. Robotun konumu ve enerjisi değişmez. Robot zaten hedefteyse HEDEFTE kaydedilir. Kutular geçilebilir hücrelerdir; GIT kutuyu otomatik almaz, teslim etmez veya şarj yapmaz. Bu eylemler açık AL, BIRAK ve SARJ komutları gerektirir.

Sonuç şeması; konum, yön, batarya, yük, teslim, çarpışma, toplam adım, enerji nedeniyle durma, en çok ziyaret edilen hücre ve hatalı satır alanlarından oluşur. En çok ziyaret alanı verilen altı örnek testte karşılaştırılmamaktadır.

C. Olay temelli kullanıcı arayüzü

Arayüz Python'un Tkinter ve ttk bileşenleriyle geliştirilmiştir [1]. Harita Canvas üzerinde ölçeklenen hücrelerle çizilir. Robot yön oku içeren bir işaretle, taşıdığı kutu ek bir göstergeyle temsil edilir. Şarj, teslim, kutu ve duvar hücreleri farklı renklere sahiptir. Ziyaret izleri açılıp kapatılabilir; fareyle hücre koordinatı ve ziyaret sayısı incelenebilir.

Bilgi paneli konum, yön, yük, batarya, teslim, çarpışma ve toplam adımı gösterir. Komut listesi sıradaki komutu vurgular. Hatalı satırlar sekmesi satır numarası, özgün içerik ve açıklamayı sunar. İşlem tablosunda her komutun sonucuna erişilir. Klasör seçimi yanında üç dosyanın ayrı ayrı seçilmesi de desteklenir.

Tümünü çalıştır işlemi, bir sonraki komutu Tk after zamanlayıcısıyla planlar. Bekleme sırasında olay döngüsü serbest kaldığı için kullanıcı duraklatabilir. Yükleme, sıfırlama ve kapatma önce bekleyen zamanlayıcıyı iptal eder. Çok büyük bir haritadaki tek GIT hesabı ise eşzamanlıdır; tek komutun hesabı süresince ekran yanıtı gecikebilir. Bu çalışmada ek thread kullanılmamıştır.

Sıfırla, dosyaları tekrar okumadan yüklenen başlangıç verisinin derin kopyasından yeni durum üretir. Üç dosya yeni oturum atanmasından önce doğrulanır. Hatalı yükleme eski oturumu bozmaz. Çalışma bittiğinde JSON ve CSV otomatik yazılır. Kullanıcı ara durumu farklı bir klasöre de kaydedebilir. JSON için standart json modülü kullanılmıştır [3].

IV. Deneysel düzen

Doğrulama, Windows üzerinde Python 3.10 ile yapılmıştır. Birinci grup, dersin sağladığı altı örnektir. İkinci grup, 22 motor ve 5 arayüz kontrolüdür. Üçüncü grup, beş deneme veri setindeki toplu/adımlı sonuç eşitliği ve sabit 217 tohumu ile üretilen 100 senaryodur. Her rastgele senaryoda 80 komut denenmiştir.

Rastgele kontroller duvara yerleşmeme, enerjinin 0 ile kapasite arasında kalması ve ziyaret toplamı değişmezini denetler. Beklenen sonuçları bulunmayan deneme verileri için bu kontroller bağımsız bir doğruluk kanıtı sayılmaz. Sonuçlar bir sonraki sayfada raporlanmaktadır.

IV. Deneysel sonuçlar

A. Otomatik kontroller

Özgün test çalıştırıcısındaki altı örnek; düz hareket, duvar çarpışması, kutu teslimi, şarj, hatalı komut satırları ve R bulunmayan haritayı kapsar. Altı örneğin tamamı geçmiştir. Test çalıştırıcısı değiştirilmemiş; öğrenci paketindeki 46 dosyanın özgün ZIP ile bayt düzeyinde aynı kaldığı doğrulanmıştır.

Kontrol grubu	Başarılı	Toplam
Sağlanan örnek test	6	6
Ek motor kontrolü	22	22
Ek Tkinter kontrolü	5	5
Rastgele senaryo	100	100

Tablo I. 26 Ekim 2026 tarihli yerel doğrulama özeti.

Ek testler, eşit enerjili işlem, yetersiz enerjide kısmi hareket, dönüşün engellenmesi, çarpışma cezası sınırı, kutu bırakıp yeniden alma ve şarj noktasına bırakmanın reddini içerir. BFS için engelli haritada altı adımlık bilinen yol, erişilemez hedef ve yol ortasında enerji yetersizliği denetlenmiştir. Arayüz testleri, sıfırlama, zamanlayıcı iptali, hatalı yüklemede eski oturumun korunması ve motor ile ekran sonuçlarının eşitliğini kontrol eder.

B. Deneme veri setleri

Beş veri setinin sonuç özeti aşağıdadır. Enerji başlangıcı ve maliyetler veri setine göre değiştiği için kalan enerji değerleri doğrudan bir performans sıralaması olarak yorumlanmamalıdır.

Veri	Konum	Enerji	Teslim	Adım
1	(14,8)	2	4	63
2	(11,1)	2	5	70
3	(10,8)	94	4	67
4	(10,1)	60	5	75
5	(13,3)	86	3	34

Tablo II. Verilen deneme verilerinin hesaplanan son durumları.

Her veri setinde bir çarpışma kaydedilmiştir. Hatalı satır sayıları sırasıyla 3, 4, 3, 5 ve 5'tir. Bütün bu çalışmalarda batarya_bitti değeri false kalmıştır. Ayrıntılı, makine tarafından okunabilir kayıt dogrulama/sonuclar.json dosyasında bulunur.

C. Tanıtım ve kapsam sınırları

Hazırlanan tanıtım senaryosu, iki teslimi ve şarj döngüsünü tek akışta gösterir. Çalışma sonunda robot (14,8) konumunda G yönündedir; 66 enerji, iki teslim, bir çarpışma ve 62 fiziksel adım kaydedilir. Komut dosyasındaki üç hatalı satır atlanır ve raporlanır.

Bu sonuçlar, daha sonra verilecek test paketinin bütün durumlarını temsil etmez. Rastgele kontroller aynı uygulamanın değişmezlerini sınar; ikinci ve bağımsız bir simülatörle karşılaştırma yapılmamıştır. Arayüz kontrolleri işlevsel akışa odaklanır. Uygulama tek robotlu, deterministik ve hücre tabanlıdır; gerçek zamanlı fizik veya otomatik görev sıralama içermez.

V. Sonuç

Depo robotunun komutlarını dosyalardan okuyup durum geçişleriyle işleyen ve sonuçlarını görselleştiren bir uygulama gerçekleştirilmiştir. Motor ile arayüzün ayrılması, otomatik test ve etkileşimli yürütmenin ortak davranmasını sağlamıştır. Kısmi hareket, yük maliyeti, çarpışma ve sıfır enerji sınırlarının açık modellenmesi, hataların izlenmesini kolaylaştırmıştır.

GIT komutu ile BFS tabanlı en kısa hücre yolu desteği eklenmiştir. Sağlanan altı örneğin ve 27 ek kontrolün geçmesi, belirtilen yerel senaryolarda gereksinimlerin karşılandığına ilişkin kanıt sunar. Sonraki geliştirmelerde çok büyük haritalarda arama işlemi küçük parçalara bölünerek arayüz yanıtı iyileştirilebilir. Enerji ağırlıklı rota seçimi ise ayrı bir hedef fonksiyonu gerektirir.

Kaynakça

[1] Python Software Foundation, “tkinter — Python interface to Tcl/Tk.” docs.python.org/3/library/tkinter.html. Erişim: 26 Ekim 2026.

[2] Python Software Foundation, “collections — Container datatypes,” deque. docs.python.org/3/library/collections.html. Erişim: 26 Ekim 2026.

[3] Python Software Foundation, “json — JSON encoder and decoder.” docs.python.org/3/library/json.html. Erişim: 26 Ekim 2026.

[4] Python Software Foundation, “csv — CSV file reading and writing.” docs.python.org/3/library/csv.html. Erişim: 26 Ekim 2026.

[5] Kocaeli Sağlık ve Teknoloji Üniversitesi, “2026-2027 Güz Dönemi Programlama Lab-I Proje 1,” ders yönergesi ve öğrenci test paketi, 2026.